

Мультиагентные системы

Gigaschool

Multi-Agent Systems vs Single-Agent: Building Reliable AI Applications

Single-Agent Systems: Simplicity & Speed

VS

Multi-Agent Systems: Specialization & Scale

What is an AI Agent?

Russell & Norvig (классическое): Агент воспринимает окружение ... и действует ... (PEAS framework)

Anthropic: Агент = автономность + tool use

OpenAI: Агент = reasoning + acting цикл

Hugging Face: Агент = AI модель, которая рассуждает, планирует и выполняет действия для достижения целей

What is an AI Agent?

Два основных типа архитектур:

- Single-Agent Systems (SAS) — одна модель, много инструментов
- Multi-Agent Systems (MAS) — много специализированных моделей, координированная работа

Single-Agent Architecture Basics

Single-Agent System (SAS) использует одну LLM с набором инструментов

User Input



[Single LLM Orchestrator]

- Читаем: запрос + описание инструментов
- Решаем: какие инструменты нужны?
- Вызываем: инструменты последовательно
- Формируем: итоговый ответ



User Output

Single-Agent Architecture Basics

Ключевые характеристики SAS:

- Unified Context — все рассуждение в одном контексте
- Single Decision-Maker — одна модель принимает все решения
- Sequential Execution — инструменты вызываются по одному
- No Agent-to-Agent Communication — нет конкурирующей координации

Примеры:

- ChatGPT с Code Interpreter и Browser
- Claude с File Upload
- Perplexity Search

Плюсы SAS

Simplicity & Efficiency

- Straightforward to design, deploy, and maintain
- Faster execution для straightforward tasks
- No messaging delays
- One reasoning chain → easy to debug

Cost-Effective

- Одна LLM
- Lower inference costs
- Меньше GPU

Best For:

- Customer support chatbots
- Code generation
- Search and summarization
- Real-time Q&A systems

Минусы SAS

- Ограничения по доменам выполняемых задач
- Ботлнеки в сложных сценариях
- Ограничения работы с контекстом
- Сложнее отлаживать галлюцинации
- Скорость работы в каскадных сценариях

Single-Agent in Practice: o3

OpenAI o3 демонстрирует максимум возможностей SAS

Почему o3 работает хорошо:

- Одна мощная модель с ризонингом
- Множество встроенных инструментов
- Обобщающая способность избыточна для бытовых задач
- Отлично справляется с декомпозицией задач
- Может проверять свои ответы в процессе генерации

Когда один агент — недостаточно

User Input (Complex Task)

↓

[Orchestrator / Coordinator]

- └─ Анализирует задачу
- └─ Разбивает на подзадачи
- └─ Определяет зависимости
- └─ Распределяет между агентами

↓

[Parallel Specialized Agents]

- └─ Agent 1 (Parser) — извлечение
- └─ Agent 2 (Analyzer) — анализ
- └─ Agent 3 (Validator) — проверка
- └─ Agent 4 (Ranker) — ранжирование

↓

[Aggregator]

- └─ Собирает и синтезирует результаты

↓

Final Output

Ключевая идея: команда экспертов
вместо одной модели

Отличия систем

Параметр	SAS	MAS
Architecture	1 LLM + Tools	N LLMs + Orchestrator
Decision-Making	Centralized	Distributed
Task Distribution	Sequential	Parallel specialization
Coordination	None needed	Complex protocols required
Complexity	Simple	Medium-High
Scalability	Limited	Easily scalable for large, dynamic environments
Context Management	Single prompt	Distributed memory

Ключевые преимущества MAS

Specialization: каждый агент имеет экспертизу в конкретном вопросе

Scalability: просто добавлять новых агентов

Fault Tolerance: система работает при падении одного из агентов

Parallelism: все агенты работают одновременно

Как работает распределение задач

SAS

Legal Document

- One LLM tries to:
 - Extract structure
 - Analyze legal terms
 - Check compliance
 - Generate summary

MAS

Legal Document

- └ Parser Agent (специализирован на extraction)
 - Raw structured data
- └ Legal Expert Agent
 - Legal analysis
- └ Compliance Agent
 - Compliance gaps
- └ Summarizer Agent
 - Executive summary

- Каждый агент может дообучен под конкретный домен
- Можно использовать разные модели (мощную для сложного, дешёвую для простого)
- Каждый агент делает что-то одно очень хорошо

Минусы MAS

Проблемы координации

- Агенты должны работать согласованно, иначе их задачи будут дублироваться
- Требуются сложные протоколы взаимодействия
- Необходимо решать вопросы маршрутизации, порядка и согласованности сообщений

Проблема потери информации

- Распределенная память = информация может теряться при передаче между агентами
- Критически важно управление контекстом

Высокие вычислительные затраты

Обслуживание и отладка

Schema-Guided Reasoning (SGR)

Schema-Guided Reasoning (SGR) — техника, которая направляет LLM к структурированному выводу через определённую схему

Три столпа SGR:

- 1) Какие шаги должен выполнить агент (obligatory reasoning checkpoints)
- 2) В каком порядке (logical flow, dependencies)
- 3) На что обратить внимание (где искать, какие поля использовать)

Technical Mechanism: Constrained Decoding

- Модель генерирует токен
- Декодер проверяет, валиден ли токен относительно схемы
- Если нет — маскируем токен

SGR Patterns: Cascade

Обеспечивает строго последовательное выполнение предопределенных шагов рассуждения

Постепенное сужение и уточнение информации, от общих размышлений к конкретному выводу.

Пример применения: Схема оценки кандидата на вакансию.

- **brief_candidate_summary**: Краткое резюме кандидата.
- **rate_skill_match**: Оценка соответствия навыков по шкале от 1 до 10.
- **final_recommendation**: Финальное решение: "нанять", "отклонить", "отложить".

Почему это работает: Поле **brief_candidate_summary** активирует в контексте модели *relevant information*, которая затем используется для более точного заполнения последующих полей.

SGR Patterns: Routing

Принуждает модель к явному выбору одного конкретного пути рассуждений из нескольких возможных

Классификация входных данных и выбор специализированной схемы действий (инструмента) для каждого случая.

Пример применения: Сортировка обращений в службу поддержки.

- **HardwareIssue:** Проблема с железом (kind: "hardware"), требует выбора component ("батарея", "экран").
- **SoftwareIssue:** Проблема с ПО (kind: "software"), требует указания *software_name*.
- **UnknownIssue:** Неизвестная проблема (kind: "unknown"), требует описания

Расширение go Tool Calling: Этот паттерн естественным образом реализует вызов инструментов. Модель выбирает, какой "инструмент" ей нужен (*SendEmailTool*, *SearchKnowledgeBaseTool*), и заполняет необходимые для его использования параметры

SGR Patterns: Cycle

Явно заставляет модель повторить шаг рассуждений несколько раз

Генерация списков или множественных оценок в рамках четко ограниченного контура.

Пример применения: Оценка рисков.

- Схема **RiskAssessment** содержит поле **factors**, которое представляет собой список элементов **RiskFactor**.
- Каждый **RiskFactor** содержит *explanation* и *severity* ("низкий", "средний", "высокий").
- Аннотации **MinLen(2)** и **MaxLen(4)** гарантируют, что модель сгенерирует от 2 до 4 факторов

Цикл можно использовать вместе с маршрутизацией, чтобы модель могла планировать несколько инструментов для выполнения сложной задачи

SGR Integration

Closed source:

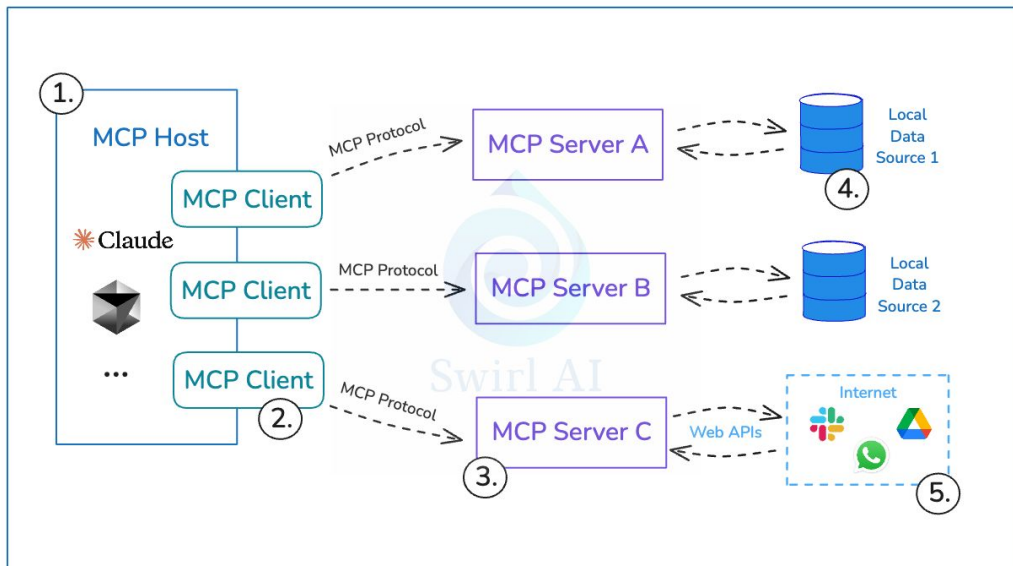
- OpenAI: `response_format="json_object"` (Structured Outputs)
- Anthropic: `response_format` parameter
- Google Gemini: `Schema` parameter

Open source:

- vLLM: Built-in Guided Decoding
- Ollama: JSON mode
- LlamaCPP: Grammar-based output

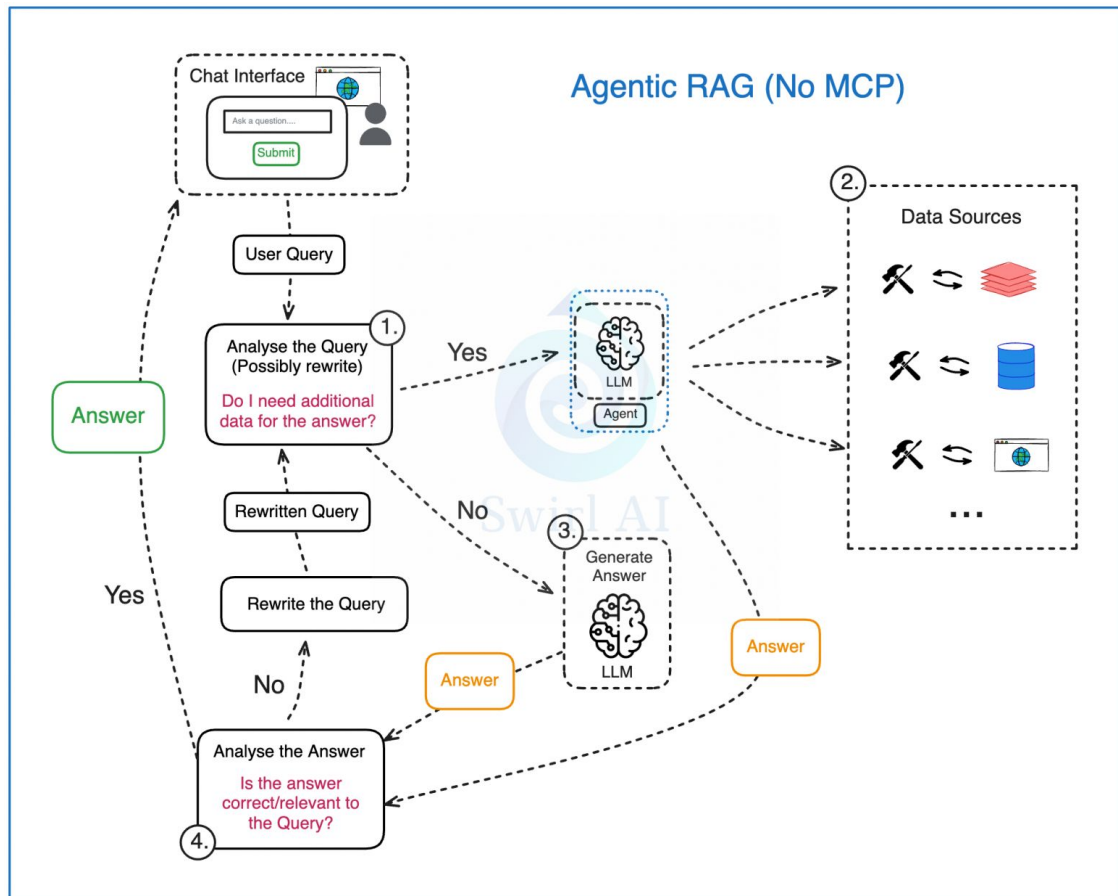
MCP (Model Context Protocol)

MCP (Model Context Protocol) — стандарт для подключения инструментов и сервисов к агентам

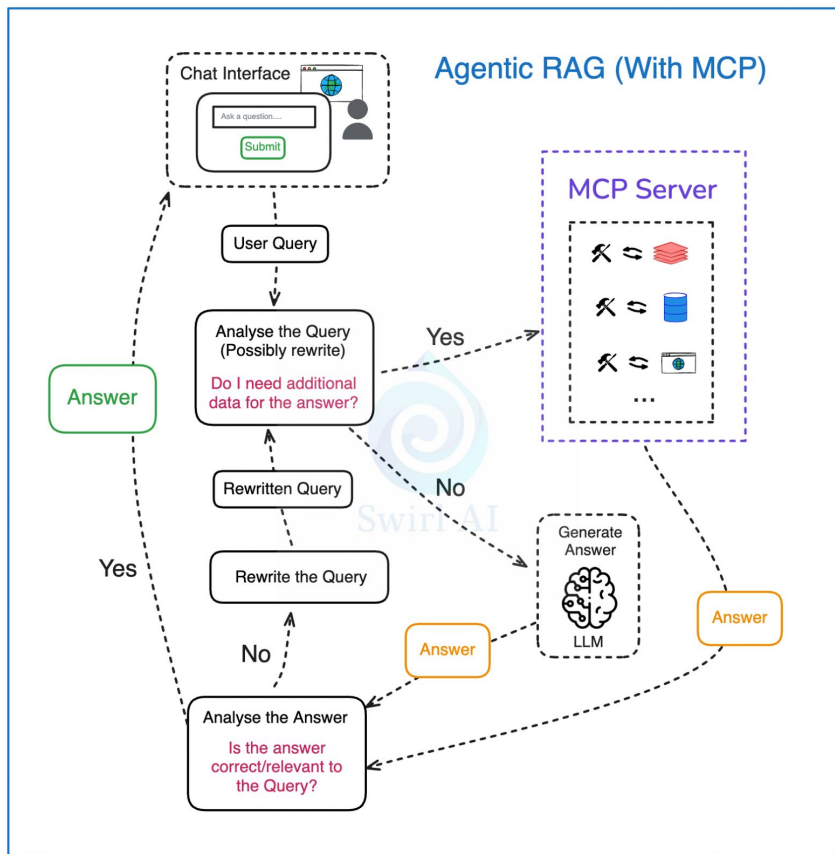


- Стандартизированный интерфейс для всех инструментов
- JSON-RPC format
- Stateless
- Легко добавлять новые серверы

В чём разница?



В чём разница?



- Раздельная логика
- Мы можем развивать компонент поиска отдельно
- Мы можем управлять безопасностью и доступом к данным через MCP-сервер
- Отдельная команда может независимо работать над данными, за которые она отвечает.

Ваши вопросы?

Полезные ссылки:

[Self-Evaluation Guided Beam Search for Reasoning](#)

[SGR Guide](#)

[Agent course](#)